PRD — Stock Research Dashboard (Saham US)

Oct 30, 2026 · @triple boys

1. Ringkasan dan tujuan

Stock Research Dashboard adalah aplikasi web pribadi untuk maksimal 5 pengguna yang memantau berita dan filing saham US di watchlist, menyaring yang penting dengan LLM, mengirim notifikasi ke Telegram, dan menampilkan scorecard laporan keuangan dari SEC EDGAR. Analisis teknikal tetap manual di TradingView.

Masalah yang diselesaikan

- Pemilik sering tertinggal berita dan filing yang relevan untuk saham yang dipegang atau diincar.
- Laporan keuangan sulit dinilai menyeluruh dan butuh pola baca yang konsisten setiap kuartal.

Tujuan (target awal, bisa diubah)

- Berita dan filing material untuk saham di watchlist sampai ke pemilik dalam 60 menit sejak terbit di sumber.
- Scorecard laporan keuangan standar dengan tren 8 kuartal, tanpa input manual.
- Biaya Rp0 untuk 5 pengguna dan sekitar 30 saham di watchlist.

Bukan tujuan

- Analisis teknikal otomatis, sinyal beli/jual, atau eksekusi transaksi.
- Saham IHSG; fase ini hanya saham US.
- Data harga real-time per tick.

Aplikasi ini alat bantu riset, bukan nasihat investasi. Setiap kesimpulan LLM harus menampilkan link sumbernya.

2. Pengguna, lingkup, dan prinsip desain

Aplikasi dipakai pemilik sebagai admin dan sampai 4 pengguna lain yang emailnya didaftarkan di allowlist. Tidak ada pendaftaran terbuka.

Peran	Hak
Admin	Mengelola allowlist email, melihat status job, mengatur watchlist sendiri
Member	Mengelola watchlist dan preferensi notifikasi sendiri, melihat berita, filing, dan scorecard

Lingkup per fase

1. Fase 1: login Google + allowlist, watchlist, pipeline berita dan filing, triase LLM, notifikasi Telegram.
2. Fase 2: data laporan keuangan dari EDGAR dan scorecard.
3. Fase 3: ringkasan LLM atas scorecard dan rilis laba, perbandingan antar saham.

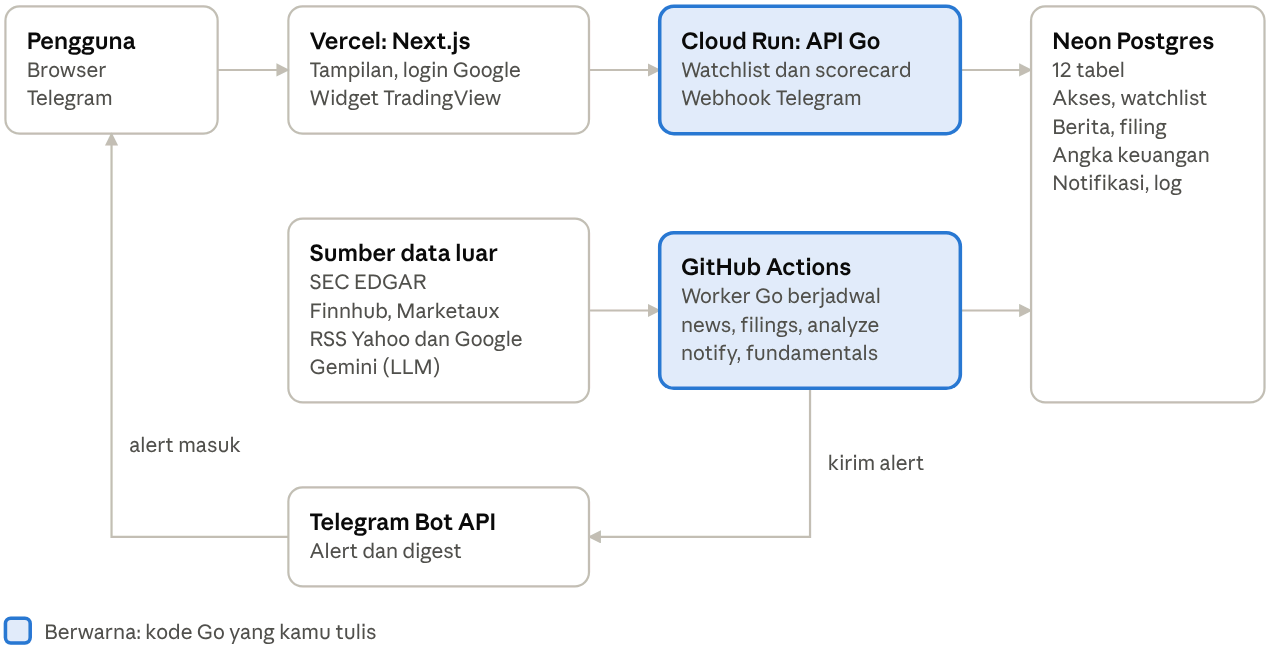
Prinsip desain

- Go menghitung semua angka; LLM hanya menjelaskan dan merangkum angka yang sudah dihitung.
- Simpan hasil olahan dan penanda posisi, bukan data mentah yang bisa diambil ulang.
- Utamakan layanan gratis, dan setiap ketergantungan pada free tier punya rencana cadangan.
- Berita disimpan sebagai judul, link, sumber, waktu, dan ringkasan; bukan isi artikel penuh.
- Satu repo, dua titik masuk Go: `cmd/api` untuk HTTP dan `cmd/worker` untuk job terjadwal.

3. Arsitektur sistem

Next.js hanya menampilkan data, sedangkan Go mengambil, menghitung, dan mengirim notifikasi.

Next.js menampilkan, Go mengambil dan mengirim



arsitektur · 6 komponen, 2 jalur

Jalur atas dipakai saat kamu membuka aplikasi: browser ke Next.js, lalu ke API Go dan database. Jalur bawah berjalan sendiri: worker Go di GitHub Actions mengambil data dari sumber luar, menyimpannya di Neon, dan mengirim alert ke Telegram. API Go dan worker memakai kode dan database yang sama.

4. Fitur dan logika

Ada empat alur inti: kelola watchlist, kumpulkan berita dan filing, saring dengan LLM lalu kirim notifikasi, dan hitung scorecard laporan keuangan. Job berjalan dari `cmd/worker`, halaman dilayani Next.js, dan semua akses data lewat API Go.

F1. Watchlist

- Pengguna menambah ticker; backend memvalidasi ke tabel `companies` (diisi dari `sec.gov/files/company_tickers.json`, diperbarui mingguan) dan menyimpan `cik`.
- Tiap item punya status `holding` atau `watching`, catatan, dan kata kunci tambahan opsional (misalnya nama produk) untuk memperluas pencarian berita.
- Batas 30 ticker per pengguna untuk menjaga kuota. Job memproses gabungan ticker unik dari semua watchlist, sekali saja per putaran.

F2. Pengumpulan berita (job `news`, tiap 30 menit)

1. Ambil ticker unik beserta kata kunci.
2. Per ticker, panggil sumber yang interval minimumnya sudah lewat (RSS 30 menit, Finnhub 120 menit, Marketaux 6 jam; lihat bagian 10), berurutan: Finnhub company news, Yahoo Finance RSS per ticker, lalu Google News RSS dengan query `"<Nama>" OR <TICKER>` dan `when:1d`. Marketaux dipakai hanya sebagai cadangan selama kuota hariannya masih ada.
3. Ubah semua hasil ke satu bentuk: ticker, judul, url, publisher, snippet, `published_at`.
4. Buang duplikat: normalisasi URL (hapus parameter `utm_*`), lalu `dedup_hash` = SHA-1 dari judul huruf kecil tanpa tanda baca ditambah publisher. Insert memakai `ON CONFLICT DO NOTHING`.
5. Simpan dengan `analyzed_at = NULL`. Saat ticker baru ditambahkan, abaikan berita yang lebih tua dari 48 jam agar tidak membanjiri notifikasi.
6. Jika satu sumber gagal (HTTP 429 atau 5xx), catat di `job_runs` dan lanjut ke sumber berikutnya; job tidak dibatalkan.

F3. Pemantauan filing SEC (job `filings`, tiap 30 menit)

1. Untuk tiap CIK, ambil `data.sec.gov/submissions/CIK##########.json` dengan header `User-Agent` berisi nama dan email, dengan laju paling banyak 5 request per detik.
2. Bandingkan `accessionNumber` dengan `companies.last_filing_seen`; proses hanya filing yang lebih baru.
3. Untuk 8-K, simpan nomor item dan beri importance awal: item 4.02 (penyajian ulang laporan keuangan) = 5; item 1.01 (perjanjian material), 2.02 (hasil operasi), dan 5.02 (perubahan pejabat) = 4; item lain = 3. Daftar ini dari pengetahuan saya, jadi cocokkan dengan daftar item 8-K di situs SEC.
4. 10-K atau 10-Q baru memicu job `fundamentals` untuk ticker itu.
5. Untuk 8-K berimportance 4 ke atas, ambil dokumen utama, potong teksnya ke sekitar 6.000 karakter, dan minta LLM meringkasnya.

F4. Triase LLM (job `analyze`, setelah `news` dan `filings`)

1. Ambil item dengan `analyzed_at IS NULL`, paling banyak 40 per putaran, dikelompokkan per ticker.

2. Satu panggilan LLM per kelompok (sampai sekitar 15 item), berisi ticker, nama perusahaan, dan daftar item bernomor (judul dan snippet saja).
3. Output wajib JSON per item: `relevan` (boolean), `kategori` (earnings, guidance, mna, regulasi_hukum, manajemen, analis, produk, makro, lainnya), `importance` 1 sampai 5, `sentimen` (positif, netral, negatif), dan `ringkasan` paling banyak 200 karakter.
4. Go memvalidasi skema JSON. Jika tidak valid, ulang sekali; jika masih gagal, pakai aturan kata kunci sebagai cadangan.
5. Rubrik importance: 5 = dapat menggerakkan harga besar atau menyangkut kelangsungan usaha (penyajian ulang laporan, akuisisi, gugatan besar, guidance berubah besar); 4 = rilis laba, pergantian CEO atau CFO, kontrak besar; 3 = perubahan rating analis, produk baru; 2 = berita sekunder; 1 = tidak relevan atau ulangan.
6. Prompt melarang menambah fakta di luar judul dan snippet; jika tidak yakin, `relevan = false` dan `importance = 2`.
7. Bila API LLM mengembalikan 429, job berhenti dan item dibiarkan `analyzed_at NULL` untuk putaran berikutnya.

F5. Notifikasi (job `notify`)

Mengikuti aturan di bagian 8: importance 4 ke atas dikirim segera, importance 3 masuk digest 07.30 WIB, dan setiap pengiriman dicatat di `notifications` dengan kunci dedup.

F6. Data laporan keuangan (job `fundamentals`, harian dan dipicu filing)

1. Untuk ticker yang 10-K atau 10-Q terbarunya belum diproses, ambil `data.sec.gov/api/xbrl/companyfacts/CIK##########.json`.
2. Peta akun di kode memetakan nama internal ke daftar nama akun XBRL menurut prioritas. Contoh: `revenue` dari `Revenues`, lalu `RevenueFromContractWithCustomerExcludingAssessedTax`, lalu `SalesRevenueNet`. Nama akun ini dari pengetahuan saya dan harus diuji pada tiap perusahaan.
3. Ambil fakta dari formulir 10-K dan 10-Q berunit USD. Satu angka sering muncul di banyak filing, jadi pilih nilai dari filing terbaru untuk periode yang sama.
4. Nilai kuartal: Q1 sampai Q3 dari 10-Q; Q4 untuk akun laba rugi dan arus kas = setahun dikurangi jumlah Q1 sampai Q3. Arus kas di 10-Q bersifat kumulatif sejak awal tahun, jadi kuartalnya adalah selisih antar periode. Akun neraca tidak perlu diturunkan: Q4 = nilai di 10-K.
5. Simpan di `fundamentals` dengan `is_derived = true` untuk nilai hasil hitungan.

6. Validasi: total aset harus sama dengan liabilitas ditambah ekuitas dalam toleransi 1%. Jika tidak, tandai scorecard `data_quality = 'suspect'` dan jangan kirim notifikasi darinya.

F7. Scorecard laporan keuangan

Go menghitung metrik berikut dari tabel `fundamentals`, dengan tren 8 kuartal. Ambang peringatan di kolom kanan adalah nilai awal usulan saya dan dapat disesuaikan.

Kelompok	Metrik	Rumus	Sinyal peringatan awal
Pertumbuhan	Pendapatan YoY	pendapatan kuartal ini dibagi kuartal yang sama tahun lalu, dikurangi 1	turun 2 kuartal berturut-turut
Pertumbuhan	EPS diluted YoY	EPS kuartal ini dibandingkan kuartal yang sama tahun lalu	EPS negatif
Profitabilitas	Margin kotor, operasional, bersih	laba kotor, laba operasi, laba bersih dibagi pendapatan	margin operasional turun lebih dari 3 poin dalam 4 kuartal
Profitabilitas	ROE (TTM)	laba bersih 4 kuartal dibagi rata-rata ekuitas	negatif
Neraca	Debt-to-equity	(utang jangka pendek + jangka panjang) dibagi ekuitas	di atas 2, atau naik 3 kuartal berturut-turut
Neraca	Current ratio	aset lancar dibagi liabilitas lancar	di bawah 1
Kualitas laba	Arus kas operasi dibanding laba bersih (TTM)	arus kas operasi dibagi laba bersih	di bawah 0,8 saat laba positif
Kualitas laba	Free cash flow	arus kas operasi dikurangi belanja modal	negatif saat laba positif
Kualitas laba	Piutang vs pendapatan	pertumbuhan piutang dikurangi pertumbuhan pendapatan	selisih lebih dari 10 poin

Kelompok	Metrik	Rumus	Sinyal peringatan awal
Dilusi	Jumlah saham diluted YoY	pertumbuhan jumlah saham rata-rata tertimbang	di atas 3%
Valuasi	PER dan harga per penjualan	harga dibagi EPS 4 kuartal; kapitalisasi dibagi penjualan 4 kuartal	di atas persentil 90 riwayat saham itu sendiri

- Tiap kelompok berstatus hijau (0 sinyal), kuning (1), atau merah (2 atau lebih). Tidak ada skor tunggal beli atau jual.
- LLM menerima angka scorecard dan daftar sinyal, lalu menulis paling banyak 150 kata: apa yang membaik, apa yang memburuk, dan apa yang perlu dicek di 10-Q berikutnya.
- Pengaman angka: setiap angka dalam teks LLM harus ada di input. Jika tidak, hasil ditolak, dibuat ulang sekali, lalu diganti templat tetap.
- Harga untuk PER diambil saat dibutuhkan dengan cache singkat, tidak disimpan sebagai histori.

F8. Halaman aplikasi dan API

Halaman: `/login`, `/watchlist`, `/ticker/[symbol]` (widget TradingView, tab Berita, Filing, Scorecard), `/feed` (semua berita importance 3 ke atas dengan filter ticker dan kategori), `/settings` (Telegram, ambang importance), dan `/admin` (allowlist, status job). Widget TradingView dipakai untuk analisis teknikal manual; data sahamnya di widget bisa tertunda tergantung bursa.

Endpoint API Go (internal, hanya dari server Next.js)	Fungsi
`GET /v1/me`	Profil dan preferensi pengguna, sekaligus pemeriksaan allowlist
`GET/POST/DELETE /v1/watchlist`	Kelola watchlist
`GET /v1/feed?ticker=&category=&min_importance=`	Daftar berita dan filing yang sudah dianalisis
`GET /v1/tickers/{symbol}/scorecard`	Scorecard terbaru dan tren

Endpoint API Go (internal, hanya dari server Next.js)	Fungsi
`POST /v1/telegram/link`	Buat token penghubung Telegram
`POST /telegram/webhook`	Menerima pesan dari Telegram (diamankan `secret_token`)
`GET/POST/PATCH /v1/admin/allowed-emails`	Kelola allowlist (admin saja)
`GET /v1/admin/job-runs`	Status job terakhir (admin saja)

5. Sumber data berita dan laporan keuangan

Laporan keuangan dan filing diambil dari SEC EDGAR (resmi, gratis, tanpa API key), sedangkan berita dikumpulkan dari tiga lapis: API berita bertag ticker, RSS, dan filing SEC itu sendiri. Batas di tabel berasal dari hasil pencarian per 30 Oktober 2026 dan belum diverifikasi di dashboard masing-masing; cek sebelum bergantung padanya.

Sumber	Dipakai untuk	Akses	Batas gratis menurut riset	Catatan
SEC EDGAR `submissions`	Filing baru: 8-K, 10-K, 10-Q, Form 4	Tanpa key; wajib header `User-Agent` berisi nama dan email	Sekitar 10 request per detik	Sumber resmi dan paling cepat
SEC EDGAR `companyfacts`	Angka laporan keuangan XBRL	Sama seperti di atas	Sama	Nama akun tidak seragam, perlu normalisasi
SEC `company_tickers.json`	Peta ticker ke CIK	Sama seperti di atas	Sama	Segarkan mingguan

Sumber	Dipakai untuk	Akses	Batas gratis menurut riset	Catatan
Finnhub company news	Berita bertag ticker	API key gratis	Sekitar 60 panggilan per menit; batas harian dilaporkan 300 sampai 1.000	Gratis untuk personal dan non-komersial; skor sentimen tidak diandalkan
Marketaux	Berita cadangan bertag ticker	API key gratis	100 request per hari	Dipakai hanya jika kuota tersisa
Yahoo Finance RSS per ticker	Berita luas per saham	Tanpa key	Sampai 20 berita per feed	Tidak resmi; sebuah situs pihak ketiga mengecek feed ini masih aktif pada 28 September 2026
Google News RSS	Berita luas, termasuk Reuters dan AP via `site:`	Tanpa key	Tidak ada batas resmi	Tidak resmi; link melewati Google News
Gemini API	Triase berita dan ringkasan	Key dari Google AI Studio	Free tier untuk model Flash dan Flash-Lite, dibatasi per project per menit dan per hari	Batas bisa berubah; data free tier dapat dipakai Google memperbaiki produknya

Sumber	Dipakai untuk	Akses	Batas gratis menurut riset	Catatan
Telegram Bot API	Notifikasi	Token bot dari BotFather	Tidak ada batas yang relevan untuk skala ini	Lihat bagian 8

Contoh endpoint

```
https://data.sec.gov/submissions/CIK0000320193.json
https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json
https://www.sec.gov/files/company_tickers.json
https://feeds.finance.yahoo.com/rss/2.0/headline?s=AAPL&region=US&lang=en-US
https://news.google.com/rss/search?q=%22Apple%22+OR+AAPL+when:1d&hl=en-US&gl=US&ceid=US:en
https://finnhub.io/api/v1/company-news?symbol=AAPL&from=2026-10-29&to=2026-10-30
```

CIK harus diisi 10 digit dengan nol di depan. Finnhub dipanggil dengan header `X-Finnhub-Token`, bukan parameter URL, supaya key tidak masuk log.

Aturan pemakaian

- SEC: selalu kirim `User-Agent`, simpan hasil di cache, batasi laju, dan hormati header `Retry-After` saat 429; tanpa itu IP bisa diblokir.
- Berita: simpan hanya judul, link, publisher, snippet, dan waktu; jangan mengunduh atau menyimpan isi artikel penuh.
- RSS tidak resmi bisa berubah atau mati sewaktu-waktu, jadi tiap sumber harus bisa gagal tanpa menghentikan job.
- Sumber berbayar yang dinormalisasi rapi (fundamental premium) sengaja tidak dipakai di fase ini.

Sumber riset

- SEC: Accessing EDGAR Data
- Panduan SEC EDGAR API
- Perbandingan API berita finansial gratis (QVeris)
- Finnhub: contoh pipeline dan batas
- Yahoo Finance RSS yang masih aktif
- Google News RSS: cara membuat dan memfilter

- Gemini API Free Tier 2026

6. Skema database

Database hanya menyimpan hal yang tidak bisa diambil ulang atau yang menghemat kuota: akses, watchlist, hasil olahan berita dan filing, angka laporan keuangan yang sudah dinormalisasi, dan catatan notifikasi. Satu project Neon (Postgres) memuat 12 tabel di bawah, disimpan sebagai `migrations/0001_init.sql` dan dijalankan dengan `goose` atau `golang-migrate`.

Kelompok	Tabel	Fungsi
Akses	`allowed_emails`, `users`, `telegram_link_tokens`	Allowlist, preferensi pengguna, penghubung Telegram
Perusahaan	`companies`, `watchlist_items`	Peta ticker ke CIK dan daftar pantauan per pengguna
Konten	`news_items`, `filings`	Berita dan filing beserta hasil triase LLM
Keuangan	`fundamentals`, `scorecards`	Angka kuartalan ternormalisasi dan hasil scorecard
Operasional	`notifications`, `source_state`, `job_runs`	Catatan kirim (anti-dobel), penanda posisi per sumber, riwayat job

```
-- ========== AKSES ==========
CREATE TABLE allowed_emails (
  email      TEXT PRIMARY KEY CHECK (email = lower(email)),
  role       TEXT NOT NULL DEFAULT 'member' CHECK (role IN
('admin','member')),
  is_active  BOOLEAN NOT NULL DEFAULT true,
  added_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email             TEXT NOT NULL UNIQUE REFERENCES allowed_emails(email) ON
UPDATE CASCADE,
  name              TEXT,
```

```
  notify_channel    TEXT NOT NULL DEFAULT 'telegram' CHECK (notify_channel IN
('telegram')),
  telegram_chat_id  BIGINT,
  min_importance    SMALLINT NOT NULL DEFAULT 4 CHECK (min_importance BETWEEN
1 AND 5),
  digest_enabled    BOOLEAN NOT NULL DEFAULT true,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_login_at     TIMESTAMPTZ
);

CREATE TABLE telegram_link_tokens (
  token       TEXT PRIMARY KEY,
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  expires_at  TIMESTAMPTZ NOT NULL,
  used_at     TIMESTAMPTZ
);

-- ========== PERUSAHAAN DAN WATCHLIST ==========
CREATE TABLE companies (
  ticker                 TEXT PRIMARY KEY,
  cik                    BIGINT NOT NULL UNIQUE,
  name                   TEXT NOT NULL,
  exchange               TEXT,
  last_filing_seen       TEXT,   -- accession number terakhir yang diproses
  last_fundamentals_accn  TEXT,   -- 10-K/10-Q terakhir yang sudah diekstrak
  updated_at             TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE watchlist_items (
  user_id   UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  ticker    TEXT NOT NULL REFERENCES companies(ticker),
  status    TEXT NOT NULL DEFAULT 'watching' CHECK (status IN
('holding','watching')),
  note      TEXT,
  keywords  TEXT[] NOT NULL DEFAULT '{}',
  added_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, ticker)
);

-- ========== BERITA DAN FILING ==========
CREATE TABLE news_items (
  id            BIGSERIAL PRIMARY KEY,
  ticker        TEXT NOT NULL REFERENCES companies(ticker),
```

```
  dedup_hash    TEXT NOT NULL,
  source        TEXT NOT NULL CHECK (source IN
('finnhub','marketaux','yahoo_rss','google_rss')),
  publisher     TEXT,
  title         TEXT NOT NULL,
  url           TEXT NOT NULL,
  snippet       TEXT,
  published_at  TIMESTAMPTZ NOT NULL,
  fetched_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  analyzed_at   TIMESTAMPTZ,
  is_relevant   BOOLEAN,
  category      TEXT,
  importance    SMALLINT CHECK (importance BETWEEN 1 AND 5),
  sentiment     TEXT CHECK (sentiment IN ('positif','netral','negatif')),
  summary       TEXT,
  UNIQUE (ticker, dedup_hash)
);
CREATE INDEX news_feed_idx      ON news_items (ticker, published_at DESC);
CREATE INDEX news_queue_idx     ON news_items (fetched_at) WHERE analyzed_at
IS NULL;
CREATE INDEX news_important_idx ON news_items (published_at DESC) WHERE
importance >= 3;

CREATE TABLE filings (
  accession_no  TEXT PRIMARY KEY,
  ticker        TEXT NOT NULL REFERENCES companies(ticker),
  form          TEXT NOT NULL,                  -- 8-K, 10-K, 10-Q, 4, ...
  filed_at      TIMESTAMPTZ NOT NULL,
  report_date   DATE,
  items         TEXT[] NOT NULL DEFAULT '{}',   -- nomor item 8-K, mis.
{2.02,9.01}
  doc_url       TEXT NOT NULL,
  importance    SMALLINT CHECK (importance BETWEEN 1 AND 5),
  summary       TEXT,
  analyzed_at   TIMESTAMPTZ
);
CREATE INDEX filings_ticker_idx ON filings (ticker, filed_at DESC);

-- ========== LAPORAN KEUANGAN ==========
CREATE TABLE fundamentals (
  ticker          TEXT NOT NULL REFERENCES companies(ticker),
  fiscal_year     INT  NOT NULL,
  fiscal_period   TEXT NOT NULL CHECK (fiscal_period IN
```

```
('Q1','Q2','Q3','Q4','FY')),
  metric          TEXT NOT NULL,                -- nama internal: revenue,
net_income, ...
  period_end      DATE NOT NULL,
  value           NUMERIC(24,4) NOT NULL,
  unit            TEXT NOT NULL DEFAULT 'USD',
  source_concept  TEXT NOT NULL,                -- akun XBRL asal
  accession_no    TEXT NOT NULL,
  is_derived      BOOLEAN NOT NULL DEFAULT false,
  PRIMARY KEY (ticker, fiscal_year, fiscal_period, metric)
);

CREATE TABLE scorecards (
  ticker        TEXT NOT NULL REFERENCES companies(ticker),
  period_end    DATE NOT NULL,
  metrics       JSONB NOT NULL,
  flags         JSONB NOT NULL DEFAULT '[]',
  group_status  JSONB NOT NULL,                 -- {"pertumbuhan":"hijau",
...}
  data_quality  TEXT NOT NULL DEFAULT 'ok' CHECK (data_quality IN
('ok','suspect')),
  llm_summary   TEXT,
  computed_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (ticker, period_end)
);

-- ========== OPERASIONAL ==========
CREATE TABLE notifications (
  id          BIGSERIAL PRIMARY KEY,
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  kind        TEXT NOT NULL CHECK (kind IN
('news_alert','filing_alert','digest','scorecard')),
  dedupe_key  TEXT NOT NULL,                    -- news:123 | filing:
<accession> | digest:2026-10-31
  channel     TEXT NOT NULL DEFAULT 'telegram',
  status      TEXT NOT NULL DEFAULT 'pending' CHECK (status IN
('pending','sent','failed')),
  attempts    SMALLINT NOT NULL DEFAULT 0,
  error       TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at     TIMESTAMPTZ,
  UNIQUE (user_id, dedupe_key)
);
```

```
CREATE INDEX notifications_pending_idx ON notifications (created_at) WHERE
status = 'pending';

CREATE TABLE source_state (
  source      TEXT NOT NULL,
  ticker      TEXT NOT NULL,
  last_cursor TEXT,
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (source, ticker)
);

CREATE TABLE job_runs (
  id           BIGSERIAL PRIMARY KEY,
  job          TEXT NOT NULL,
  started_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  finished_at  TIMESTAMPTZ,
  status       TEXT NOT NULL DEFAULT 'running' CHECK (status IN
('running','ok','partial','failed')),
  stats        JSONB,
  error        TEXT
);
CREATE INDEX job_runs_idx ON job_runs (job, started_at DESC);
```

Pembersihan berkala (dijalankan job `maintenance` harian)

```
DELETE FROM news_items WHERE published_at < now() - interval '90 days';
DELETE FROM job_runs   WHERE started_at  < now() - interval '30 days';
DELETE FROM telegram_link_tokens WHERE expires_at < now() - interval '1 day';
```

Catatan desain

- Satu berita yang menyangkut dua ticker disimpan sebagai dua baris, karena relevansi dan importance dinilai per ticker.
- `notifications.dedupe_key` dengan `UNIQUE (user_id, dedupe_key)` adalah pengaman utama agar satu berita tidak terkirim dua kali walau job berjalan tumpang tindih.
- Nilai `notify_channel` saat ini hanya `telegram`; menambah kanal berarti mengubah `CHECK` lewat migrasi baru.
- Perkiraan kasar saya: untuk 30 ticker dan retensi 90 hari, berita dan hasil triase berada di puluhan MB, dan angka keuangan dalam satuan MB, jauh di bawah jatah gratis sekitar 0,5 sampai 1 GB per project Neon.

7. Login Google dan allowlist email

Login hanya lewat Google OAuth memakai Auth.js di Next.js, dan email yang tidak ada di tabel `allowed_emails` ditolak saat sign-in. Tidak ada form daftar atau password sendiri.

Alur login

1. Pengguna menekan Masuk dengan Google dan Auth.js mengarahkannya ke Google.
2. Google mengembalikan profil; callback `signIn` memeriksa `email_verified` bernilai true dan email (huruf kecil) terdaftar dan aktif di `allowed_emails`, lewat API Go.
3. Tidak terdaftar: tolak dan tampilkan halaman Akses tidak diizinkan. Terdaftar: buat atau perbarui baris `users`, lalu terbitkan sesi JWT di cookie httpOnly.
4. Browser tidak memanggil API Go langsung. Server Next.js yang memanggil Go dengan header `X-Internal-Key` dan `X-User-Email`.
5. Go membandingkan kunci dengan perbandingan waktu-konstan dan memeriksa ulang email di allowlist pada setiap request. Email yang dicabut kehilangan akses meski JWT-nya belum kedaluwarsa.

Mengelola allowlist

- Admin pertama dimasukkan lewat SQL: `INSERT INTO allowed_emails (email, role) VALUES ('emailmu@gmail.com', 'admin');`
- Halaman Admin menambah email atau menonaktifkannya (`is_active = false`); data pengguna yang dinonaktifkan tetap tersimpan.
- Semua email disimpan dan dibandingkan dalam huruf kecil.

Konfigurasi Google Cloud

1. Di Google Cloud Console buat project, lalu buka APIs & Services, OAuth consent screen. Pilih tipe External, isi nama aplikasi dan email dukungan, dan batasi scope ke `openid`, `email`, `profile`.
2. Buat Credentials, OAuth client ID, tipe Web application. Isi Authorized redirect URI dengan `http://localhost:3000/api/auth/callback/google` dan `https://<domain-vercel>/api/auth/callback/google`.
3. Salin Client ID dan Client Secret ke variabel Vercel `AUTH_GOOGLE_ID` dan `AUTH_GOOGLE_SECRET`. Buat `AUTH_SECRET` dengan `openssl rand -base64 32`.
4. Status publikasi consent screen: mode Testing hanya mengizinkan akun yang didaftarkan sebagai test user. Untuk scope dasar, dari pengetahuan saya bisa dipindah ke In production tanpa verifikasi Google; cek di konsol. Keamanan tetap ditentukan allowlist kita, bukan status ini.

Catatan keamanan: cookie sesi `Secure` dan `HttpOnly`, masa sesi 30 hari, `INTERNAL_API_KEY` hanya ada di environment server Vercel dan Cloud Run, dan tabel `users` hanya menyimpan preferensi, bukan status login.

8. Notifikasi: Telegram sebagai kanal utama

Notifikasi dikirim lewat Telegram, karena WhatsApp resmi tidak gratis untuk pesan yang kita kirim lebih dulu. Alert aplikasi ini selalu dimulai dari sistem, jadi memakai template WhatsApp yang berbayar per pesan, sedangkan Bot API Telegram gratis (dari pengetahuan saya, bukan dari halaman yang dibuka).

Opsi	Biaya	Syarat dan risiko	Keputusan
Telegram Bot API	Gratis	Buat bot lewat BotFather; pengguna menekan Start sekali	Dipakai
WhatsApp Cloud API (resmi)	Sandbox pengembangan gratis; produksi ditagih per pesan	Akun Meta Business, nomor sendiri, dan template disetujui untuk pesan di luar jendela 24 jam	Ditunda
WhatsApp tidak resmi (Baileys, whatsapp-web.js, WAHA)	Gratis	Melanggar ketentuan, ada risiko nomor diblokir, dan butuh proses yang selalu menyala dengan sesi tersimpan	Tidak dipakai

Alasan WhatsApp resmi ditunda. Menurut halaman harga Meta, pesan non-template hanya boleh dikirim dalam jendela 24 jam setelah pengguna membalas. Untuk menghubungi lebih dulu, bisnis wajib memakai template. Sejak 1 Oktober 2026, pesan layanan di dalam jendela gratis hanya untuk 1.000 pesan per nomor per bulan, dan template utility yang dikirim dalam jendela juga ditagih. Tarif per pasar (termasuk Indonesia, dalam IDR) ada di rate card Meta, yang tidak saya salin ke sini. Opsi tidak resmi gratis, tapi tidak cocok dengan Cloud Run yang tidur dan berisiko menghilangkan nomormu.

Desain agar WhatsApp bisa ditambah nanti

- Satu antarmuka Go `Notifier` dengan metode `Send(ctx, user, message)`; implementasi pertama `TelegramNotifier`.
- Kolom `users.notify_channel` dan `notifications.channel` mencatat kanal yang dipakai.
- Logika kapan mengirim (aturan di bawah) tidak bergantung pada kanal.

Menghubungkan Telegram ke akun

1. Di halaman Pengaturan, pengguna menekan Hubungkan Telegram; backend membuat token sekali pakai berlaku 15 menit.
2. Pengguna membuka `https://t.me/<nama_bot>?start=<token>`.
3. Webhook `POST /telegram/webhook` di API Go menerima `/start <token>`, mencocokkan token, menyimpan `telegram_chat_id`, lalu membalas konfirmasi.
4. Webhook didaftarkan dengan `setWebhook` dan `secret_token`; API menolak request tanpa header rahasia itu.

Aturan pengiriman

- Importance 4 atau 5 (atau ambang `users.min_importance`): kirim segera, satu pesan per item.
- Importance 3: masuk digest harian pukul 07.30 WIB berisi ringkasan semalam.
- Importance 1 atau 2: hanya tampil di dashboard.
- Batas 10 alert per jam per pengguna; kelebihannya digabung jadi satu pesan ringkas.
- Kunci dedup `news:<id>` atau `filing:<accession>` di tabel `notifications` mencegah kirim ganda.
- Gagal kirim: ulang 3 kali dengan jeda bertambah, lalu tandai `failed` dan masuk log job.

Format pesan: satu baris judul dengan ticker, kategori dan importance, ringkasan satu sampai dua kalimat, nama sumber, dan link ke artikel atau filing.

9. Panduan deployment

Urutan pemasangan: database, bot dan kunci API, backend Go, frontend, webhook Telegram, penjadwal, lalu uji. Semua layanan memakai paket gratis; angka batasnya ada di bagian 10.

Struktur repo

```
stock-dashboard/
  web/                      # Next.js (App Router)
  api/
    cmd/api/main.go         # server HTTP
    cmd/worker/main.go      # job: news, filings, analyze, notify, digest,
                            #      fundamentals, sync-companies, maintenance
    internal/               # sources, llm, scoring, notify, store
    migrations/
    Dockerfile
  .github/workflows/jobs.yml
```

Prasyarat: akun GitHub, Neon, Google Cloud (billing aktif), Vercel, dan Telegram; serta key untuk Gemini (Google AI Studio), Finnhub, dan Marketaux (opsional).

Langkah 1. Database di Neon

1. Daftar di neon.com, lalu Create project bernama `stock-dashboard`. Pilih region Asia Pasifik terdekat yang tersedia, sedekat mungkin dengan region Cloud Run.
2. Dari Connection Details salin dua string: yang pooled (host mengandung `-pooler`) untuk API dan worker, dan yang direct untuk migrasi. Bentuknya `postgres://user:pass@host/db?sslmode=require`.
3. Jalankan migrasi: `migrate -path api/migrations -database "$DATABASE_URL_DIRECT" up` (golang-migrate), atau pakai `goose`.
4. Daftarkan admin pertama: `psql "$DATABASE_URL_DIRECT" -c "INSERT INTO allowed_emails(email, role) VALUES ('emailmu@gmail.com','admin');"`
5. Isi tabel `companies` dengan menjalankan `go run ./cmd/worker -jobs=sync-companies` dari folder `api` (setelah variabel lingkungan diatur).

Database tidur setelah sekitar 5 menit tanpa query dan bangun otomatis pada query berikutnya, dengan jeda sesaat.

Langkah 2. Bot Telegram dan kunci API

1. Di Telegram buka @BotFather, kirim `/newbot`, isi nama dan username (berakhiran `bot`). Simpan token yang diberikan sebagai `TELEGRAM_BOT_TOKEN`.
2. Buat dua string acak: `openssl rand -hex 32` untuk `INTERNAL_API_KEY` dan `openssl rand -hex 24` untuk `TELEGRAM_WEBHOOK_SECRET`.

3. Buat key Gemini di Google AI Studio, key Finnhub di finnhub.io, dan key Marketaux (opsional) di marketaux.com. Pilih nilai `LLM_MODEL` dari model Flash atau Flash-Lite yang tertulis gratis di halaman harga Gemini saat kamu memasang.
4. Tentukan `SEC_USER_AGENT` berisi nama aplikasi dan emailmu, misalnya `StockDash emailmu@gmail.com`.

Langkah 3. Backend Go di Cloud Run

Dockerfile di `api/Dockerfile` (samakan `GO_VERSION` dengan `go.mod`; server wajib mendengarkan port dari variabel `PORT`):

```
ARG GO_VERSION=1.23
FROM golang:${GO_VERSION} AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12
COPY --from=build /out/api /api
USER nonroot
ENTRYPOINT ["/api"]
```

1. Pasang gcloud CLI, lalu `gcloud auth login` dan `gcloud config set project <ID_PROJECT>`.
2. Aktifkan API: `gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com secretmanager.googleapis.com`.
3. Simpan rahasia di Secret Manager, satu per satu: `printf '%s' "$NILAI" | gcloud secrets create DATABASE_URL --data-file=-`. Ulangi untuk `INTERNAL_API_KEY`, `TELEGRAM_BOT_TOKEN`, dan `TELEGRAM_WEBHOOK_SECRET`.
4. Beri akses baca rahasia ke akun layanan bawaan Cloud Run:

```
PROJECT=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT" --
format='value(projectNumber)')
gcloud projects add-iam-policy-binding "$PROJECT" \
  --member="serviceAccount:${PROJECT_NUMBER}-
compute@developer.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
```

5. Deploy dari source (Cloud Build membuat image dari Dockerfile):

```
gcloud run deploy stock-api \
  --source api \
  --region REGION_ANDA \
  --allow-unauthenticated \
  --min-instances 0 --max-instances 2 \
  --memory 256Mi --cpu 1 \
  --set-secrets
DATABASE_URL=DATABASE_URL:latest,INTERNAL_API_KEY=INTERNAL_API_KEY:latest,TEL
EGRAM_BOT_TOKEN=TELEGRAM_BOT_TOKEN:latest,TELEGRAM_WEBHOOK_SECRET=TELEGRAM_WE
BHOOK_SECRET:latest \
  --set-env-vars TELEGRAM_BOT_USERNAME=nama_botmu_bot
```

6. Catat URL layanan yang ditampilkan di akhir. Uji: `curl https://<url>/healthz` harus mengembalikan `ok`.

Catatan penting:

- `--allow-unauthenticated` dipakai karena Vercel memanggil dari internet; perlindungannya adalah `INTERNAL_API_KEY` dan pemeriksaan allowlist di tiap request.
- `--min-instances 0` membuat layanan tidur saat tidak dipakai (request pertama lebih lambat sekitar 1 sampai 5 detik) dan membuatnya masuk jatah gratis. `--max-instances 2` membatasi biaya.
- Pilih region yang masuk free tier. Halaman harga Cloud Run memuat daftar region yang dikecualikan dari free tier, jadi cek sebelum memilih; Singapura umumnya dipilih demi latensi dari Indonesia, tapi periksa tier-nya.
- Pasang budget alert di Billing, Budgets & alerts (misalnya 1 dan 5 USD) dengan notifikasi email. Alert hanya memberi tahu dan tidak menghentikan tagihan.
- Cloud Build dan Artifact Registry punya jatah gratis sendiri; hapus image lama secara berkala agar tetap di bawahnya.

Langkah 4. Frontend di Vercel

1. Dorong repo ke GitHub.
2. Di Vercel pilih Add New, Project, impor repo, dan atur Root Directory ke `web` (framework Next.js terdeteksi otomatis).
3. Isi Environment Variables untuk Production dan Preview: `AUTH_SECRET`, `AUTH_GOOGLE_ID`, `AUTH_GOOGLE_SECRET`, `API_BASE_URL` (URL Cloud Run), dan `INTERNAL_API_KEY`.
4. Deploy. Catat domain `*.vercel.app`, lalu tambahkan `https://<domain>/api/auth/callback/google` ke Authorized redirect URI di OAuth client Google (bagian 7).
5. Uji: login dengan email admin berhasil, dan login dengan email yang tidak terdaftar ditolak.

Paket Hobby Vercel ditujukan untuk pemakaian pribadi non-komersial (dari pengetahuan saya; baca ketentuan terbarunya).

Langkah 5. Hubungkan Telegram

```
curl "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/setWebhook" \
  -d "url=https://<url-cloud-run>/telegram/webhook" \
  -d "secret_token=$TELEGRAM_WEBHOOK_SECRET"
```

Lalu di aplikasi buka Pengaturan, Hubungkan Telegram, ikuti link, tekan Start, dan pastikan bot membalas konfirmasi.

Langkah 6. Penjadwal dengan GitHub Actions

Tambahkan rahasia di repo (Settings, Secrets and variables, Actions): `DATABASE_URL`, `GEMINI_API_KEY`, `LLM_MODEL`, `FINNHUB_API_KEY`, `MARKETAUX_API_KEY`, `TELEGRAM_BOT_TOKEN`, dan `SEC_USER_AGENT`. Cron GitHub memakai UTC, jadi 00.30 UTC adalah 07.30 WIB. Menit tidak dibuat tepat di awal jam agar tidak berebut dengan antrean GitHub.

```
name: jobs
on:
  schedule:
    - cron: '7,37 * * * *'   # news, filings, analyze, notify
    - cron: '30 0 * * *'     # digest 07.30 WIB
    - cron: '23 2 * * *'     # fundamentals, maintenance
```

```
    - cron: '41 3 * * 1'     # sync-companies mingguan
  workflow_dispatch:
    inputs:
      job:
        description: 'Nama job, dipisah koma'
        required: true

concurrency:
  group: jobs
  cancel-in-progress: false

jobs:
  run:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version-file: api/go.mod
          cache-dependency-path: api/go.sum
      - name: Pilih job
        id: pick
        env:
          SCHEDULE: ${{ github.event.schedule }}
          INPUT_JOB: ${{ inputs.job }}
        run: |
          case "$SCHEDULE" in
            '7,37 * * * *') JOBS='news,filings,analyze,notify' ;;
            '30 0 * * *')   JOBS='digest' ;;
            '23 2 * * *')   JOBS='fundamentals,maintenance' ;;
            '41 3 * * 1')   JOBS='sync-companies' ;;
            *)              JOBS="$INPUT_JOB" ;;
          esac
          echo "jobs=$JOBS" >> "$GITHUB_OUTPUT"
      - name: Jalankan worker
        working-directory: api
        env:
          JOBS: ${{ steps.pick.outputs.jobs }}
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
          GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
          LLM_MODEL: ${{ secrets.LLM_MODEL }}
          FINNHUB_API_KEY: ${{ secrets.FINNHUB_API_KEY }}
```

```
          MARKETAUX_API_KEY: ${{ secrets.MARKETAUX_API_KEY }}
          TELEGRAM_BOT_TOKEN: ${{ secrets.TELEGRAM_BOT_TOKEN }}
          SEC_USER_AGENT: ${{ secrets.SEC_USER_AGENT }}
        run: go run ./cmd/worker -jobs="$JOBS"
```

Batas menit Actions. Repo private mendapat 2.000 menit per bulan. Perkiraan saya, satu run memakan 1 sampai 2 menit (termasuk kompilasi), sehingga interval 30 menit sepanjang hari (sekitar 1.440 run per bulan) bisa melewati kuota. Pilihannya: jadikan repo public (menit tanpa batas; pastikan tidak ada rahasia di kode), ubah cron menjadi tiap jam, atau batasi jam jalan. Ada laporan komunitas bahwa jadwal cron di repo private pada akun gratis kadang tidak berjalan, padahal dokumentasi menyebut 2.000 menit; karena itu uji dulu (Langkah 7).

Rencana B bila Actions tidak cocok: jalankan worker sebagai Cloud Run Job dan picu dengan Cloud Scheduler. Cloud Run Jobs punya jatah gratis sendiri, dan Cloud Scheduler gratis untuk beberapa job per akun billing; cek jumlahnya di halaman harga.

Langkah 7. Uji menyeluruh

- [ ] Login admin berhasil, email di luar allowlist ditolak.
- [ ] Tambah AAPL ke watchlist; `companies` berisi AAPL.
- [ ] Jalankan job `news` lewat tombol Run workflow; baris `news_items` bertambah.
- [ ] Jalankan `analyze`; kolom `importance` terisi dan validasi JSON lolos.
- [ ] Turunkan `min_importance` ke 1 untuk uji; pesan Telegram masuk satu kali, tidak dobel di putaran berikutnya.
- [ ] Jalankan `fundamentals` untuk AAPL; scorecard muncul dan `data_quality = 'ok'`.
- [ ] Cron berjalan sendiri dalam 1 jam (cek tab Actions).
- [ ] Budget alert Google Cloud aktif; `max-instances` terpasang.

Variabel lingkungan

Variabel	Dipakai oleh	Isi
`DATABASE_URL`	api, worker	Connection string Neon yang pooled
`DATABASE_URL_DIRECT`	migrasi lokal	Connection string Neon yang direct

Variabel	Dipakai oleh	Isi
`INTERNAL_API_KEY`	web, api	String acak panjang; Next.js mengirimnya ke Go
`AUTH_SECRET`	web	Hasil `openssl rand -base64 32`
`AUTH_GOOGLE_ID`, `AUTH_GOOGLE_SECRET`	web	Dari OAuth client Google
`API_BASE_URL`	web	URL layanan Cloud Run
`TELEGRAM_BOT_TOKEN`	api, worker	Token dari BotFather
`TELEGRAM_WEBHOOK_SECRET`	api	String acak untuk memeriksa webhook
`TELEGRAM_BOT_USERNAME`	api	Username bot tanpa tanda @
`GEMINI_API_KEY`, `LLM_MODEL`	worker	Key AI Studio dan nama model Flash atau Flash-Lite yang gratis
`FINNHUB_API_KEY`	worker	Untuk berita dan harga (PER)
`MARKETAUX_API_KEY`	worker	Opsional, berita cadangan
`SEC_USER_AGENT`	worker	Nama aplikasi dan email, wajib untuk SEC

10. Batas free tier, risiko, dan mitigasi

Dua batas paling dekat dengan pemakaian nyata adalah menit GitHub Actions dan kuota harian Finnhub; selebihnya aman dengan margin besar. Kolom pemakaian adalah perkiraan saya untuk 30 ticker, bukan hasil pengukuran.

Layanan	Jatah gratis (hasil riset)	Perkiraan pemakaian	Status
Neon, compute	100 CU-jam per bulan per project, tidur setelah 5 menit idle	Sekitar 36 CU-jam: job tiap 30 menit membangunkan database sekitar 6 menit per run	Aman

Layanan	Jatah gratis (hasil riset)	Perkiraan pemakaian	Status
Neon, storage	Sekitar 0,5 sampai 1 GB per project	Puluhan MB dengan retensi 90 hari	Aman
Cloud Run	2 juta request, 180.000 vCPU-detik, 360.000 GiB-detik per bulan	Beberapa ribu request ringan per bulan	Aman
GitHub Actions (repo private)	2.000 menit per bulan	Sekitar 1.440 run per bulan pada interval 30 menit, 1 sampai 2 menit per run	Berisiko
Finnhub	Sekitar 60 panggilan per menit; batas harian dilaporkan 300 sampai 1.000	30 ticker × 48 putaran = 1.440 panggilan per hari bila dipanggil tiap 30 menit	Berisiko
Gemini free tier	Dibatasi per project per menit dan per hari; angkanya ada di halaman rate limit AI Studio	Satu panggilan per kelompok ticker yang punya berita baru	Pantau
Marketaux	100 request per hari	Hanya cadangan, di bawah 20 per hari	Aman

Penyesuaian akibat tabel di atas: setiap sumber punya interval minimum sendiri di `source_state`, yaitu RSS 30 menit, Finnhub 120 menit, dan Marketaux 6 jam. Job tetap berjalan tiap 30 menit tetapi melewati sumber yang belum waktunya.

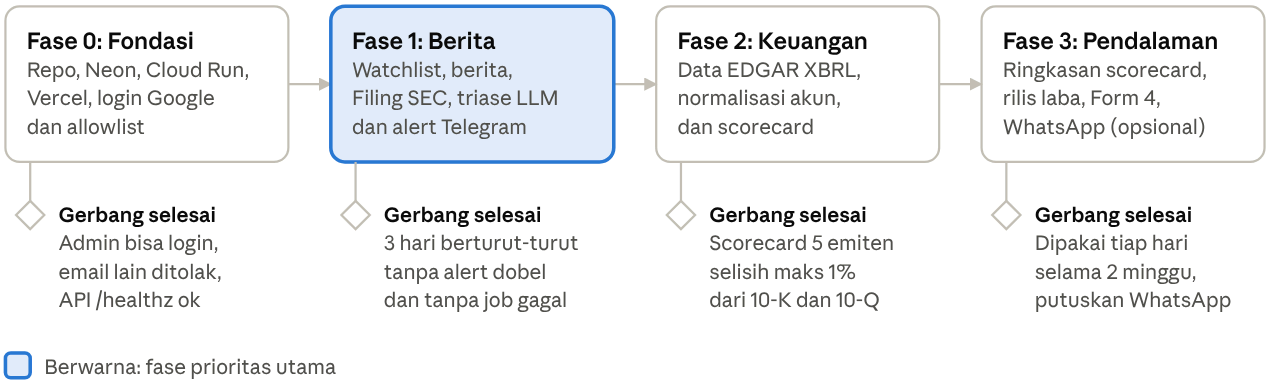
Risiko	Dampak	Mitigasi
Batas atau ketentuan free tier berubah (contoh: kuota Gemini dipangkas pada Desember 2025)	Sebagian fitur berhenti mendadak	Setiap sumber dan LLM berada di balik antarmuka Go; ada cadangan aturan kata kunci; kegagalan dicatat di `job_runs`
RSS tidak resmi (Yahoo, Google News) berubah atau memblokir IP datacenter	Berita berkurang	Banyak sumber paralel, jeda 1 detik antar request, dan notifikasi admin bila satu sumber gagal 3 putaran berturut-turut

Risiko	Dampak	Mitigasi
Jadwal GitHub Actions telat, tidak jalan, atau menit habis	Notifikasi terlambat atau berhenti	Repo public, atau interval tiap jam, atau Cloud Scheduler (rencana B); peringatan ke admin bila `job_runs` terakhir lebih dari 2 jam
LLM salah menilai atau mengarang	Notifikasi sampah atau berita penting terlewat	Rubrik tetap, validasi skema JSON, ringkasan hanya dari judul dan snippet, link sumber selalu tampil, ambang importance bisa diubah
Angka XBRL salah atau tidak lengkap	Scorecard menyesatkan	Validasi aset = liabilitas + ekuitas, `data_quality`, uji awal pada beberapa emiten besar, tampilkan nomor accession sumber
Biaya tak terduga di Google Cloud	Tagihan tak diharapkan	`--max-instances 2`, budget alert, hapus image lama di Artifact Registry
Rahasia bocor	Akses ke data dan kuota API	Semua rahasia di Secret Manager dan GitHub Secrets; jangan commit file `.env`; rotasi key bila ragu
Hak cipta dan ketentuan situs berita	Risiko hukum dan pemblokiran	Simpan hanya metadata dan ringkasan, selalu tautkan ke sumber, pakai untuk pribadi
Email yang dicabut masih memegang sesi aktif	Akses lama tetap terbuka	Go memeriksa allowlist pada setiap request, bukan hanya saat login
Hasil dianggap nasihat investasi	Keputusan keliru	Label bukan nasihat investasi di tiap halaman; tidak ada keluaran beli atau jual

11. Roadmap dan kriteria selesai

Berita dan notifikasi dikerjakan lebih dulu karena itu masalah utamamu; laporan keuangan menyusul setelah alert berjalan stabil.

Berita dan notifikasi dulu, laporan keuangan menyusul



roadmap · 4 fase, 4 gerbang

Gerbang di bawah tiap fase adalah usulan awal kriteria selesai, termasuk angka seperti 3 hari dan 5 emiten. Ubah sesuai kebutuhanmu sebelum mulai mengerjakan fase berikutnya.